```
// Code-A1 (Fibonacci)
#include
using namespace std;
int recursiveSteps = 0;
int iterativeSteps = 0;
// Recursive function for Fibonacci
int fibRecursive(int n) {
recursiveSteps++;
if (n <= 1)
return n;
return fibRecursive(n - 1) + fibRecursive(n - 2);
}
// Non-recursive (Iterative) Fibonacci
void fibIterative(int n) {
if (n <= 0)
return;
int prev = 0, curr = 1, next;
cout << prev << " ";
if (n == 1)
return;
cout << curr <<" ";
for (int i = 2; i < n; i++) {
next = prev + curr;
cout << next <<" ";
prev = curr;
curr = next;
iterativeSteps++;
}
}
int main() {
int n;
cout <<"Enter the number of terms: ";
cin >> n;
cout <<"\nFibonacci Series using Recursion: ";
for (int i = 0; i < n; i++) {
cout << fibRecursive(i) <<" ";
}
cout <<"\nTotal Recursive Steps: " << recursiveSteps;
cout << "\n\nFibonacci Series using Iteration:";
fibIterative(n);
cout << "\nTotal Iterative Steps:" << iterativeSteps;
cout << endl;
return 0;
}
```

```
// SAMPLE OUTOUT
/
Enter the number of terms: 5

Fibonacci Series using Recursion: 0 1 1 2 3
* Total Recursive Steps: 19

Fibonacci Series using Iteration: 0 1 1 2 3
* Total Iterative Steps: 3
*/
explain the code
```

This C++ program generates the **Fibonacci series using two methods:**

1. Recursion
2. Iteration (without recursion)

It also counts and compares the number of steps performed by both methods.

Let's understand it line by line in simple language for your DAA practical and viva.

1. What is the Fibonacci series?

The Fibonacci series is a sequence in which every number is the sum of the previous two numbers.

Example:

```
0 1 1 2 3 5 8 13 21 34
```

Formula:

$$F(n) = F(n-1) + F(n-2)$$

- First number = 0
- Second number = 1
- Every next number = Sum of previous two numbers.

2. Explanation of the code

A. Header file and global variables

```
#include <iostream>
using namespace std;

int recursiveSteps = 0;
int iterativeSteps = 0;
```

- `#include <iostream>`: Used for input and output operations.

- `using namespace std`: Allows us to use `cout` and `cin` without writing `std::`.
- `recursiveSteps`: Counts how many times the recursive function is called.
- `iterativeSteps`: Counts how many times the iterative loop calculates a new Fibonacci number.

B. Recursive Fibonacci function

```
int fibRecursive(int n) {
    recursiveSteps++;

    if (n <= 1)
        return n;

    return fibRecursive(n - 1) + fibRecursive(n - 2);
}
```

This function calculates Fibonacci numbers using recursion.

Explanation:

1. `recursiveSteps++`: Increases the recursive function call counter by 1.
2. `if (n <= 1)`: Checks the base condition.
3. `return n`: If n is 0 or 1, return the same number.
4. `fibRecursive(n-1) + fibRecursive(n-2)`: Calls the same function again to calculate the previous two Fibonacci numbers.

Example:

To calculate `fibRecursive(4)`:

```
fib(4)
   |
   |---- fib(3)
   |       |---- fib(2)
   |       |       |---- fib(1) = 1
   |       |       |---- fib(0) = 0
   |       |
   |       |---- fib(1) = 1
   |
   |---- fib(2)
           |---- fib(1) = 1
           |---- fib(0) = 0
```

Result = 3.

C. Iterative Fibonacci function

```
void fibIterative(int n) {
    if (n <= 0)
        return;
```

```
    int prev = 0, curr = 1, next;

    cout << prev << " ";

    if (n == 1)
        return;

    cout << curr << " ";

    for (int i = 2; i < n; i++) {
        next = prev + curr;

        cout << next << " ";

        prev = curr;
        curr = next;

        iterativeSteps++;
    }
}
```

This function calculates Fibonacci numbers using a loop instead of calling itself.

Explanation:

- `if (n <= 0)`: If the number of terms is zero or negative, stop.
- `prev = 0`: Stores the previous Fibonacci number.
- `curr = 1`: Stores the current Fibonacci number.
- `next`: Stores the next Fibonacci number.
- `cout << prev`: Prints the first number (0).
- `cout << curr`: Prints the second number (1).

For loop:

```
for (int i = 2; i < n; i++)
```

The loop starts from 2 because the first two Fibonacci numbers are already printed.

```
next = prev + curr;
```

Calculates the next Fibonacci number.

```
prev = curr;
curr = next;
```

Updates the previous and current numbers for the next iteration.

```
iterativeSteps++;
```

Increases the iteration counter.

D. Main function

```
int main() {
    int n;

    cout << "Enter the number of terms: ";
    cin >> n;
```

- `main()` is the starting point of the program.
- `int n` stores the number of Fibonacci terms.
- `cin >> n` takes input from the user.

Printing Fibonacci using recursion:

```
cout << "\nFibonacci Series using Recursion: ";

for (int i = 0; i < n; i++) {
    cout << fibRecursive(i) << " ";
}
```

The loop calls the recursive function for every number from 0 to n-1.

Printing recursive steps:

```
cout << "\nTotal Recursive Steps: " << recursiveSteps;
```

Displays the total number of recursive function calls.

Printing Fibonacci using iteration:

```
cout << "\n\nFibonacci Series using Iteration: ";

fibIterative(n);
```

Calls the iterative function to generate the Fibonacci series.

Printing iterative steps:

```
cout << "\nTotal Iterative Steps: " << iterativeSteps;
```

Displays the total number of iterative calculations.

3. Sample output

PROGRAM OUTPUT

```
Enter the number of terms: 5

Fibonacci Series using Recursion: 0 1 1 2 3
Total Recursive Steps: 19

Fibonacci Series using Iteration: 0 1 1 2 3
Total Iterative Steps: 3
```

Both methods produce the same series

Why are the steps 19 and 3?

For recursion, the function is called:

Fibonacci number	Function calls
fib(0)	1
fib(1)	1
fib(2)	3
fib(3)	5
fib(4)	9
Total	**19**

For iteration, the loop calculates three new numbers:

- 1st iteration: 1 + 0 = 1
- 2nd iteration: 1 + 1 = 2
- 3rd iteration: 2 + 1 = 3

Therefore, iterative steps = 3.

4. Time and space complexity

Method	Time complexity	Auxiliary space
Recursion	$O(2^n)$	O(n)
Iteration	O(n)	O(1)

Why?

- Recursion repeatedly calculates the same Fibonacci numbers, so it takes more time.

- Iteration calculates each number only once, so it is faster.
- Recursion uses a function call stack.
- Iteration uses only a few variables.

5. Important viva questions and answers

Q1. What is Fibonacci series?

A. It is a series where each number is the sum of the previous two numbers.

Q2. What is recursion?

A. Recursion is a process in which a function calls itself.

Q3. What is iteration?

A. Iteration is a process of repeating statements using loops like `for` and `while`.

Q4. What is the base condition in this program?

A. `if (n <= 1) return n;` is the base condition that stops recursion.

Q5. Which method is faster?

A. Iteration is faster because it avoids repeated calculations.

Q6. Why do we use `recursiveSteps++`?

A. To count how many times the recursive function is called.

Q7. Why do we use `iterativeSteps++`?

A. To count the number of new Fibonacci numbers calculated inside the loop.

Q8. What is the difference between recursion and iteration?

A. Recursion uses function calls, whereas iteration uses loops.

Q9. What is the time complexity of recursive Fibonacci?

A. $O(2^n)$.

Q10. What is the time complexity of iterative Fibonacci?

A. O(n).

Important for practical viva

Remember these 5 points:

1. Fibonacci means the sum of the previous two numbers.

2. Recursion means a function calls itself.
3. Iteration means using loops.
4. Recursive Fibonacci has $O(2^n)$ time complexity.
5. Iterative Fibonacci has O(n) time complexity.

One important observation: Both methods give the same output, but iteration is more efficient because it avoids repeated calculations.

```
// Code-A2 (Huffman Coding)
#include
#include
#include
#include
using namespace std;
// A Huffman Tree node
struct Node {
char ch;
int freq;
Node left, right;
Node(char c, int f) {
ch = c;
freq = f;
left = right = nullptr;
}
};
// Compare function for priority queue (min-heap)
struct Compare {
bool operator()(Node* a, Node* b) {
return a->freq > b->freq; // smaller frequency => higher priority
}
};
// Recursive function to print Huffman Codes
void printCodes(Node* root, string code) {
if (!root) return;
// Leaf node -> contains a character
if (!root->left && !root->right)
cout << root->ch << " : " << code << endl;
printCodes(root->left, code + "0");
printCodes(root->right, code + "1");
}
// Main function to build Huffman Tree and generate codes
void huffmanEncoding(vector& chars, vector& freq) {
priority_queue<Node, vector<Node>, Compare> pq;
// Step 1: Create a leaf node for each character and add it to the queue
for (int i = 0; i < chars.size(); i++)
```

```
pq.push(new Node(chars[i], freq[i]));
// Step 2: Repeat until one node remains (the root)
while (pq.size() > 1) {
Node* left = pq.top(); pq.pop();
Node* right = pq.top(); pq.pop();
// Create new internal node with sum of two smallest frequencies
Node* newNode = new Node('-', left->freq + right->freq);
newNode->left = left;
newNode->right = right;
pq.push(newNode);
}
// Step 3: Print the Huffman codes
Node* root = pq.top();
cout << "\nHuffman Codes:\n";
printCodes(root, "");
}
int main() {
int n;
cout <<"Enter number of characters: ";
cin >> n;
vector chars(n);
vector freq(n);
cout <<"Enter characters: ";
for (int i = 0; i < n; i++)
cin >> chars[i];
cout <<"Enter corresponding frequencies: ";
for (int i = 0; i < n; i++)
cin >> freq[i];
huffmanEncoding(chars, freq);
return 0;
}
// SAMPLE OUTPUT
/
$ ./a.out
* Enter number of characters: 5
* Enter characters: a f v c w
* Enter corresponding frequencies: 3 2 1 5 7

Huffman Codes:
* w : 0
* c : 10
* a : 110
* v : 1110
* f : 1111
*/
```

This C++ program implements **Huffman Coding**, a data compression technique used to reduce the number of bits required to store data.

It uses a **Greedy Algorithm** to generate unique binary codes for each character based on its frequency.

Let's understand it line by line in simple language for your DAA practical and viva.

1. What is Huffman Coding?

Huffman Coding is a lossless data compression technique.

It assigns:

- Shorter binary codes to characters that occur more frequently.
- Longer binary codes to characters that occur less frequently.

Example:

Suppose we have these characters:

Character	Frequency
a	3
f	2
v	1
c	5
w	7

Here, `w` occurs most frequently, so it gets a shorter code.

The purpose is to reduce the total number of bits needed to represent the data.

2. Explanation of the code

A. Header files

```
#include <iostream>
#include <queue>
#include <vector>
#include <string>
using namespace std;
```

- `iostream`: Used for input and output (`cin`, `cout`).
- `queue`: Used to implement the priority queue.
- `vector`: Used to store characters and their frequencies.
- `string`: Used to store binary Huffman codes.

- `using namespace std`: Avoids writing `std::` repeatedly.

B. Creating a Huffman Tree node

```
struct Node {
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f) {
        ch = c;
        freq = f;
        left = right = nullptr;
    }
};
```

This structure represents one node of the Huffman Tree.

It contains:

- `ch`: Stores the character.
- `freq`: Stores the frequency of that character.
- `left`: Pointer to the left child.
- `right`: Pointer to the right child.

Constructor:

```
Node(char c, int f) {
    ch = c;
    freq = f;
    left = right = nullptr;
}
```

It initializes the character and frequency.

Initially, both child pointers are set to `nullptr` because the node has no children.

C. Compare function (Min-Heap)

```
struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};
```

This function decides the priority of nodes in the priority queue.

- Node with smaller frequency gets higher priority.
- Node with larger frequency gets lower priority.

For example:

Character	Frequency
v	1
f	2
a	3
c	5
w	7

The node with frequency 1 will be removed first.

Remember: This priority queue works as a min-heap.

D. Recursive function to print Huffman codes

```
void printCodes(Node* root, string code) {
    if (!root)
        return;

    if (!root->left && !root->right)
        cout << root->ch << " : " << code << endl;

    printCodes(root->left, code + "0");
    printCodes(root->right, code + "1");
}
```

This function traverses the Huffman Tree and generates binary codes.

Explanation:

1. `if (!root) return;`

   Checks whether the node is empty. If it is empty, stop.

2. `if (!root->left && !root->right)`

   Checks whether the current node is a leaf node.

3. `cout << root->ch << " : " << code`

   Prints the character and its Huffman code.

4. `printCodes(root->left, code + "0")`

   Moves to the left child and adds `0` to the code.

5. `printCodes(root->right, code + "1")`

   Moves to the right child and adds `1` to the code.

Important rule:

- Left side = 0
- Right side = 1

E. Huffman Encoding function

This is the main logic of the program.

```
void huffmanEncoding(vector<char>& chars, vector<int>& freq) {
    priority_queue<Node*, vector<Node*>, Compare> pq;
```

Creates a priority queue using a min-heap.

It stores pointers to Huffman Tree nodes and arranges them according to frequency.

Step 1: Create leaf nodes

```
for (int i = 0; i < chars.size(); i++)
    pq.push(new Node(chars[i], freq[i]));
```

Creates a node for every character and inserts it into the priority queue.

For example:

```
a = 3
f = 2
v = 1
c = 5
w = 7
```

Step 2: Build the Huffman Tree

```
while (pq.size() > 1) {
    Node* left = pq.top();
    pq.pop();

    Node* right = pq.top();
    pq.pop();
```

- `pq.top()` gets the node with the smallest frequency.
- `pq.pop()` removes that node from the queue.
- The two smallest frequency nodes are selected.

Next:

```
Node* newNode = new Node('-', left->freq + right->freq);
newNode->left = left;
newNode->right = right;

pq.push(newNode);
```

- Creates a new internal node.
- Adds the frequencies of the two selected nodes.
- Connects them as left and right children.
- Inserts the new node back into the priority queue.

This process continues until only one node remains.

That final node becomes the root of the Huffman Tree.

Step 3: Print Huffman codes

```
Node* root = pq.top();

cout << "\nHuffman Codes:\n";
printCodes(root, "");
```

- `pq.top()` gets the final root node.
- `printCodes()` traverses the tree.
- It generates and prints the binary code for every character.

F. Main function

```
int main() {
    int n;

    cout << "Enter number of characters: ";
    cin >> n;
```

Takes the total number of characters from the user.

```
vector<char> chars(n);
vector<int> freq(n);
```

Creates two vectors:

- `chars`: Stores characters.
- `freq`: Stores their corresponding frequencies.

```
for (int i = 0; i < n; i++)
    cin >> chars[i];
```

Accepts the characters.

```
for (int i = 0; i < n; i++)
    cin >> freq[i];
```

Accepts their frequencies.

```
huffmanEncoding(chars, freq);
```

Calls the Huffman encoding function.

```
return 0;
```

Terminates the program successfully.

3. Dry run of the program

Input:

```
Number of characters: 5
Characters: a f v c w
Frequencies: 3 2 1 5 7
```

The algorithm repeatedly combines the two smallest frequencies.

Step	Selected frequencies	New frequency
1	v(1) + f(2)	3
2	a(3) + (vf)(3)	6
3	c(5) + (avf)(6)	11
4	w(7) + (cavf)(11)	18

One valid Huffman Tree gives these codes:

Character	Frequency	Huffman code
w	7	0
c	5	10
a	3	110
v	1	1110
f	2	1111

SAMPLE OUTPUT

```
Enter number of characters: 5
Enter characters: a f v c w
Enter corresponding frequencies: 3 2 1 5 7

Huffman Codes:
w : 0
c : 10
a : 110
v : 1110
f : 1111
```

Note: If two nodes have equal frequencies, their ordering can vary depending on the priority queue. The exact codes may differ, but the compression cost remains optimal.

4. Time and space complexity

Let n be the number of characters.

Operation	Complexity
Creating leaf nodes	O(n log n)
Building Huffman Tree	O(n log n)
Printing Huffman codes	O(n) approximately
Overall time complexity	O(n log n)
Space complexity	O(n)

5. Important viva questions and answers

Q1. What is Huffman Coding?

Ans: Huffman Coding is a lossless data compression technique that assigns variable-length binary codes to characters based on their frequency.

Q2. Which algorithm is used in Huffman Coding?

Ans: Greedy Algorithm.

Q3. Why do we use a priority queue?

Ans: To efficiently select the two nodes with the smallest frequencies.

Q4. What is a min-heap?

Ans: A min-heap is a data structure in which the smallest element has the highest priority.

Q5. What is a Huffman Tree?

Ans: It is a binary tree used to generate Huffman codes for characters.

Q6. What is a leaf node?

Ans: A node that does not have any left or right child.

Q7. Why do we assign 0 to the left and 1 to the right?

Ans: To generate unique binary codes for each character. The left/right assignment can be reversed as long as it is consistent.

Q8. What is lossless compression?

Ans: Compression in which the original data can be recovered without losing any information.

Q9. What is the time complexity of Huffman Coding?

Ans: O(n log n).

Q10. Why do frequently occurring characters get shorter codes?

Ans: To reduce the total number of bits required to store the data.

Q11. What is a greedy approach?

Ans: A method that makes the best possible local choice at each step to achieve an optimal solution.

Q12. Where is Huffman Coding used?

Ans: ZIP compression, file compression, JPEG encoding and data compression systems.

---

6. Most important points to remember for your practical

1. Huffman Coding is a lossless compression technique.
2. It uses a Greedy Algorithm.
3. It uses a Priority Queue (Min-Heap).
4. Two smallest frequencies are combined at every step.
5. Left edge represents 0 and right edge represents 1.
6. Frequent characters get shorter codes.
7. Time complexity is O(n log n).

Test your understanding

DAA Viva Practice · 0/4 answered

1. Which algorithm does Huffman Coding use?

○ Dynamic Programming
○ Greedy Algorithm
○ Divide and Conquer
○ Backtracking

Incorrect. Correct answer: Greedy Algorithm

Huffman Coding repeatedly selects the two smallest frequencies, which is a greedy choice.

2. Which data structure is used to select the two smallest frequencies?

○ Stack
○ Array
○ Min-Heap Priority Queue
○ Linked List

Incorrect. Correct answer: Min-Heap Priority Queue

The min-heap priority queue efficiently provides the smallest frequency nodes.

3. What does a leaf node represent?

○ A character
○ The sum of all frequencies
○ An empty node
○ A loop

Incorrect. Correct answer: A character

Each original character is stored in a leaf node.

4. What is the time complexity?

○ O(n)
○ $O(n^2)$
○ O(log n)
○ O(n log n)

Incorrect. Correct answer: O(n log n)

Priority queue operations take O(log n), repeated O(n) times.

```
// Code-A3 (Fractical Knapsack)
#include
#include
#include
using namespace std;
// Structure for an item
struct Item {
int weight;
int value;
};
// Comparison function to sort items by value/weight ratio
bool compare(Item a, Item b) {
double r1 = (double)a.value / a.weight;
double r2 = (double)b.value / b.weight;
return r1 > r2; // Sort in descending order
}
double fractionalKnapsack(int W, vector& items) {
// Sort items by value-to-weight ratio
sort(items.begin(), items.end(), compare);
double totalValue = 0.0; // Total value in knapsack
for (auto &i : items) {
if (W == 0) break; // Knapsack is full
// If item can be fully taken
if (i.weight <= W) {
W -= i.weight;
totalValue += i.value;
} else {
// Take fraction of item
totalValue += i.value * ((double)W / i.weight);
W = 0;
}
}
return totalValue;
}
int main() {
int n, W;
cout << “Enter number of items:”;
cin >> n;
vector items(n);
cout << “Enter value and weight of each item:\n”;
for (int i = 0; i < n; i++)
cin >> items[i].value >> items[i].weight;
cout << “Enter capacity of knapsack:”;
cin >> W;
double maxValue = fractionalKnapsack(W, items);
```

```
cout << “\nMaximum value in knapsack =” << maxValue << endl;
return 0;
}
// SAMPLE OUTPUT
/
$ ./a.out
* Enter number of items: 3
* Enter value and weight of each item:
* 12 32
* 56 67
* 23 87
* Enter capacity of knapsack: 45

Maximum value in knapsack = 37.6119
*/
```

This C++ program implements the **Fractional Knapsack Problem** using the **Greedy Algorithm**.

It finds the maximum total value that can be placed inside a knapsack (bag) with limited capacity. Unlike 0/1 Knapsack, here we can take a fraction of an item.

Let’s understand it line by line in simple language for your DAA practical and viva.

1. What is the Fractional Knapsack Problem?

Suppose you have a bag with a maximum capacity of 45 kg and three items.

Item	Value	Weight
A	12	32 kg
B	56	67 kg
C	23	87 kg

Our objective is to select items so that we get the **maximum possible value without exceeding the bag’s capacity**.

In Fractional Knapsack:

- We can take the complete item.
- We can take a fraction of an item.
- We cannot exceed the knapsack capacity.

Main formula:

$$Ratio = \frac{Value}{Weight}$$

We select items with the highest value-to-weight ratio first.

2. Explanation of the code

A. Header files

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
```

- `iostream`: Used for input and output (`cin`, `cout`).
- `vector`: Used to store multiple items.
- `algorithm`: Used for sorting items.
- `using namespace std`: Avoids writing `std::` repeatedly.

B. Structure for an item

```
struct Item {
    int weight;
    int value;
};
```

This structure stores the information of each item.

It contains two variables:

- `weight`: Weight of the item.
- `value`: Value or profit of the item.

For example:

```
Item A:
Weight = 32
Value = 12
```

C. Comparison function

```
bool compare(Item a, Item b) {
    double r1 = (double)a.value / a.weight;
    double r2 = (double)b.value / b.weight;

    return r1 > r2;
}
```

This function compares two items based on their value-to-weight ratio.

Explanation:

1. `r1` calculates the ratio of the first item.
2. `r2` calculates the ratio of the second item.
3. `return r1 > r2` sorts items in descending order.

This means the item with the highest ratio comes first.

Example:

Item	Value	Weight	Ratio
A	12	32	0.375
B	56	67	0.836
C	23	87	0.264

After sorting:

1. B = 0.836
2. A = 0.375
3. C = 0.264

D. Fractional Knapsack function

```
double fractionalKnapsack(int W, vector<Item>& items) {
```

This function calculates the maximum value that can fit inside the knapsack.

- `W`: Maximum capacity of the knapsack.
- `items`: Stores all the items.
- `double`: Returns the maximum value, which can be a decimal number.

Step 1: Sort items

```
sort(items.begin(), items.end(), compare);
```

Sorts all items according to their value-to-weight ratio.

Highest ratio comes first.

Step 2: Initialize total value

```
double totalValue = 0.0;
```

Stores the total value collected inside the knapsack.

Initially, it is zero.

Step 3: Traverse all items

```
for (auto &i : items) {
    if (W == 0)
        break;
```

- `for (auto &i : items)`: Visits each item one by one.
- `if (W == 0)`: Checks whether the bag is full.
- `break`: Stops the loop if the capacity becomes zero.

Step 4: Check whether the complete item fits

```
if (i.weight <= W) {
    W -= i.weight;
    totalValue += i.value;
}
```

If the complete item fits inside the knapsack:

- Subtract its weight from the remaining capacity.
- Add its complete value to the total value.

Example:

Capacity = 45 kg.

Suppose an item weighs 20 kg and has value 30.

After taking it:

- Remaining capacity = 45 - 20 = 25 kg.
- Total value = 30.

Step 5: Take a fraction of the item

```
else {
    totalValue += i.value * ((double)W / i.weight);
    W = 0;
}
```

If the complete item cannot fit, we take only the fraction that fits.

Formula:

$$Fraction = \frac{Remaining\ Capacity}{Item\ Weight}$$

$$Value\ Added = Item\ Value \times Fraction$$

After taking the fraction, the knapsack becomes full.

Step 6: Return maximum value

```
return totalValue;
```

Returns the final maximum value obtained.

E. Main function

```
int main() {
    int n, W;
```

- `n`: Number of items.
- `W`: Knapsack capacity.

Taking number of items:

```
cout << "Enter number of items: ";
cin >> n;
```

Accepts the number of items from the user.

Creating a vector:

```
vector<Item> items(n);
```

Creates a vector to store the details of all items.

Taking item values and weights:

```
for (int i = 0; i < n; i++)
    cin >> items[i].value >> items[i].weight;
```

Accepts the value and weight of every item.

Taking knapsack capacity:

```
cout << "Enter capacity of knapsack: ";
cin >> W;
```

Accepts the maximum weight the bag can hold.

Calling the function:

```
double maxValue = fractionalKnapsack(W, items);
```

Calls the Fractional Knapsack function and stores the maximum value.

Displaying the result:

```
cout << "\nMaximum value in knapsack = " << maxValue << endl;
```

Prints the final maximum value.

3. Dry run of the program

Input:

```
Number of items: 3

Value  Weight
12     32
56     67
23     87

Knapsack capacity: 45
```

First, calculate the value-to-weight ratio.

Item	Value	Weight	Ratio
A	12	32	0.375
B	56	67	0.836
C	23	87	0.264

After sorting, the order is B, A, C.

Selection process:

Step	Selected item	Weight taken	Value added	Remaining capacity
1	B	45 (fraction)	37.6119	0

For item B:

$$Fraction = \frac{45}{67} = 0.67164$$

$$Value = 56 \times \frac{45}{67}$$

$$Value = 37.6119$$

FINAL OUTPUT

```
Enter number of items: 3

Enter value and weight of each item:
12 32
56 67
23 87

Enter capacity of knapsack: 45

Maximum value in knapsack = 37.6119
```

Maximum value = 37.6119

4. Time and space complexity

Let n be the number of items.

Operation	Complexity
Sorting items	O(n log n)
Traversing items	O(n)
Calculating fractions	O(1) per item
Overall time complexity	O(n log n)
Auxiliary space	O(log n) typically

Why O(n log n)?

Because sorting takes O(n log n), and traversing the items takes O(n). Sorting dominates the overall time complexity.

5. Difference between Fractional and 0/1 Knapsack

This is a very important viva question.

Fractional Knapsack	0/1 Knapsack
Items can be divided	Items cannot be divided
Fractions are allowed	Only complete items
Uses Greedy Algorithm	Usually uses Dynamic Programming
Time complexity O(n log n) for this implementation	Standard DP: O(nW)
Selects based on value/weight ratio	Considers combinations of items

6. Important viva questions and answers

Q1. What is the Fractional Knapsack Problem?

Ans: It is an optimization problem in which we select items to maximize total value without exceeding the knapsack capacity. We can take fractions of items.

Q2. Which algorithm is used?

Ans: Greedy Algorithm.

Q3. What is a Greedy Algorithm?

Ans: It makes the best possible choice at each step to achieve an optimal solution.

Q4. What is the formula for value-to-weight ratio?

Ans:

$$Ratio = \frac{Value}{Weight}$$

Q5. Why do we sort items in descending order?

Ans: To select items having the highest value per unit weight first.

Q6. Why is the return type of the function double?

Ans: Because taking a fraction of an item can produce a decimal value.

Q7. What is the purpose of the compare function?

Ans: It sorts the items in descending order based on their value-to-weight ratio.

Q8. What happens when the knapsack becomes full?

Ans: The loop stops using the `break` statement.

Q9. Can we take half of an item?

Ans: Yes. Fractional Knapsack allows us to take any fraction of an item.

Q10. What is the time complexity?

Ans: O(n log n), because of sorting.

Q11. Is Greedy always optimal for 0/1 Knapsack?

Ans: No. Greedy is optimal for Fractional Knapsack, but not always for 0/1 Knapsack.

Q12. Where is the Knapsack Problem used?

Ans: Resource allocation, cargo loading, budget optimization and investment planning.

---

7. Important points to remember for practical viva

1. Fractional Knapsack is an optimization problem.
2. It uses the Greedy Algorithm.
3. Items are sorted using value/weight ratio.
4. Highest ratio is selected first.
5. Fractions of items are allowed.
6. `sort()` is used to arrange items.
7. `double` is used because the answer can be decimal.
8. Time complexity = O(n log n).

Test your understanding

DAA Viva Practice

0/4 answered

1. Which algorithm is used in Fractional Knapsack?

○ Greedy
○ Dynamic Programming
○ Backtracking
○ BFS

Correct answer: Greedy

Greedy selects the highest value-to-weight ratio first.

2. What is the formula used to select items?

○ Weight / Value
○ Value × Weight
○ Value / Weight
○ Value + Weight

Correct answer: Value / Weight

The ratio tells us the value obtained per unit of weight.

3. Can we take a fraction of an item?

○ No
○ Yes
○ Only half
○ Only if weight is zero

Correct answer: Yes

Fractional Knapsack permits partial items.

4. What is the time complexity of this program?

○ O(n)
○ $O(n^2)$
○ O(n log n)
○ $O(2^n)$

Correct answer: O(n log n)

Sorting dominates the overall time complexity.

```
// Code-A4 (0-1 Knapsack)
#include
#include
using namespace std;
int knapsack(int W, vector& wt, vector& val, int n) {
// dp[i][w] = max value for first i items with weight limit w
vector<vector> dp(n + 1, vector(W + 1, 0));
// Build the table dp[][] bottom-up
for (int i = 1; i <= n; i++) {
for (int w = 1; w <= W; w++) {
if (wt[i - 1] <= w)
dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
else
dp[i][w] = dp[i - 1][w];
}
}
return dp[n][W]; // The bottom-right cell gives the result
}
int main() {
int n, W;
cout << “Enter number of items:”;
cin >> n;
vector val(n), wt(n);
cout << “Enter values of items:\n”;
for (int i = 0; i < n; i++) cin >> val[i];
cout << “Enter weights of items:\n”;
for (int i = 0; i < n; i++) cin >> wt[i];
cout << “Enter capacity of knapsack:”;
cin >> W;
int maxValue = knapsack(W, wt, val, n);
cout << “\nMaximum value in knapsack =” << maxValue << endl;
return 0;
}
// SAMPLE OUTPUT
/
$ ./a.out
* Enter number of items: 3
* Enter values of items:
* 12
* 32
* 14
* Enter weights of items:
* 45
* 34
* 65
```

```
* Enter capacity of knapsack: 60

Maximum value in knapsack = 32
*/
```

This C++ program implements the **0/1 Knapsack Problem using Dynamic Programming (DP).**

It finds the maximum value that can fit inside a bag with limited capacity, where each item can either be selected completely or not selected at all.

Let's understand it line by line in simple language for your DAA practical and viva.

1. What is the 0/1 Knapsack Problem?

Suppose you have a bag with a maximum capacity of 60 kg and three items.

Item	Value	Weight
A	12	45 kg
B	32	34 kg
C	14	65 kg

Our objective is to select items to get the maximum possible value without exceeding the bag's capacity.

Why is it called 0/1 Knapsack?

- `1` means we select the complete item.
- `0` means we do not select the item.
- We cannot take half or a fraction of an item.

2. Explanation of the code

A. Header files

```
#include <iostream>
#include <vector>
using namespace std;
```

- `iostream`: Used for input and output (`cin`, `cout`).
- `vector`: Used to store item values, weights and the DP table.
- `using namespace std`: Avoids writing `std::` repeatedly.

B. Knapsack function

```
int knapsack(int W, vector<int>& wt, vector<int>& val, int n) {
```

This function calculates the maximum value that can fit inside the knapsack.

Parameters:

- `W`: Maximum capacity of the bag.
- `wt`: Vector containing item weights.
- `val`: Vector containing item values.
- `n`: Total number of items.

The return type is `int` because the program uses integer values.

C. Creating the DP table

```
vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
```

This creates a two-dimensional DP table.

- Rows represent the number of items considered.
- Columns represent the available knapsack capacity.
- Initially, all values are zero.

For example, if `n = 3` and `W = 60`, the table has:

- 4 rows (0 to 3).
- 61 columns (0 to 60).

Why do we use n+1 and W+1?

Because we also consider:

- Zero items.
- Zero capacity.

Both situations give a maximum value of zero.

D. Building the DP table

```
for (int i = 1; i <= n; i++) {
    for (int w = 1; w <= W; w++) {
```

We use two nested loops.

- Outer loop: Selects items one by one.
- Inner loop: Checks every possible capacity from 1 to W.

This is called the bottom-up approach of Dynamic Programming.

E. Checking whether the item fits

```
if (wt[i - 1] <= w)
```

Checks whether the current item's weight is less than or equal to the available capacity.

Why `i-1`?

Because vector indexing starts from zero, but the DP table starts from one for the first item.

F. Main DP formula

```
dp[i][w] = max(
    val[i - 1] + dp[i - 1][w - wt[i - 1]],
    dp[i - 1][w]
);
```

This is the most important line in the program.

We have two choices:

Choice 1: Include the item

```
val[i - 1] + dp[i - 1][w - wt[i - 1]]
```

- Add the value of the current item.
- Calculate the best value possible using the remaining capacity.
- Consider only the previous items.

Choice 2: Exclude the item

```
dp[i - 1][w]
```

Do not select the current item. Keep the maximum value obtained from previous items.

```
max(choice1, choice2)
```

Select whichever option gives the higher value.

If the item does not fit:

```
else
    dp[i][w] = dp[i - 1][w];
```

If the item's weight exceeds the available capacity, we cannot select it. Therefore, we copy the previous result.

G. Returning the answer

```
return dp[n][W];
```

Returns the bottom-right cell of the DP table.

This cell contains the maximum value possible using all items and the full knapsack capacity.

3. Main function

```
int main() {
    int n, W;
```

Declares:

- `n`: Number of items.
- `W`: Capacity of the knapsack.

Taking the number of items:

```
cout << "Enter number of items: ";
cin >> n;
```

Accepts the number of items.

Creating vectors:

```
vector<int> val(n), wt(n);
```

Creates two vectors:

- `val`: Stores values.
- `wt`: Stores weights.

Taking item values:

```
for (int i = 0; i < n; i++)
    cin >> val[i];
```

Accepts the value of each item.

Taking item weights:

```
for (int i = 0; i < n; i++)
    cin >> wt[i];
```

Accepts the weight of each item.

Taking knapsack capacity:

```
cin >> W;
```

Accepts the maximum capacity.

Calling the function:

```
int maxValue = knapsack(W, wt, val, n);
```

Calls the knapsack function and stores the maximum value.

Displaying the result:

```
cout << "\nMaximum value in knapsack = " << maxValue << endl;
```

Prints the final answer.

4. Dry run of the program

Input:

```
Number of items: 3

Values:
12 32 14

Weights:
45 34 65

Knapsack capacity: 60
```

Let’s check each item.

Item	Value	Weight	Can fit in 60 kg?
A	12	45	Yes
B	32	34	Yes
C	14	65	No

Possible combinations:

Selected items	Total weight	Total value
A	45	12
B	34	32
C	65	Not allowed

Selected items	Total weight	Total value
A + B	79	Not allowed

Therefore, the best choice is item B.

Maximum value = 32

Understanding the DP table

For this example, the important capacity values are 34, 45 and 60.

Items considered	Capacity 34	Capacity 45	Capacity 60
No items	0	0	0
A (12, 45)	0	12	12
B (32, 34)	32	32	32
C (14, 65)	32	32	32

The final answer at `dp[3][60]` is 32.

5. Time and space complexity

Let:

- `n` = Number of items.
- `W` = Knapsack capacity.

Complexity	Value
Time complexity	O(n × W)
Space complexity	O(n × W)

Why O(n × W)?

Because there are two nested loops:

- Outer loop runs n times.
- Inner loop runs W times.

Why O(n × W) space?

Because we create a two-dimensional DP table with `(n+1) × (W+1)` elements.

6. Difference between 0/1 and Fractional Knapsack

Very important for DAA viva.

0/1 Knapsack	Fractional Knapsack
Items cannot be divided	Items can be divided
Complete item or no item	Fractions are allowed
Uses Dynamic Programming	Uses Greedy Algorithm
Time: O(nW) for this DP implementation	Time: O(n log n)
Uses a DP table	Uses sorting and selection

7. Important viva questions and answers

Q1. What is the 0/1 Knapsack Problem?

Ans: It is an optimization problem where we select items to maximize total value without exceeding the knapsack capacity. Each item can either be selected or rejected.

Q2. Which algorithm is used?

Ans: Dynamic Programming.

Q3. What is Dynamic Programming?

Ans: Dynamic Programming is a technique that solves a complex problem by dividing it into smaller overlapping subproblems and storing their results.

Q4. Why is it called 0/1 Knapsack?

Ans: Because each item has only two choices: 0 means not selected and 1 means selected.

Q5. Why do we use a 2D array?

Ans: To store the maximum value for different numbers of items and different knapsack capacities.

Q6. What is the recurrence relation?

Ans:

$$DP[i][w] = \max \begin{cases} val[i-1] + DP[i-1][w - wt[i-1]] \\ DP[i-1][w] \end{cases}$$

This applies when the item fits.

Q7. What happens if the item weight is greater than capacity?

Ans: We exclude the item and copy the previous DP value.

Q8. What is the bottom-up approach?

Ans: It solves smaller subproblems first and uses their results to solve larger problems.

Q9. What is the time complexity?

Ans: O(n × W).

Q10. Can we take half of an item in 0/1 Knapsack?

Ans: No. We must take the complete item or reject it.

Q11. What is the base condition?

Ans: When the number of items or knapsack capacity is zero, the maximum value is zero.

Q12. Why do we use the max() function?

Ans: To select the better result between including and excluding the current item.

---

8. Important points to remember for practical viva

1. 0/1 Knapsack is an optimization problem.
2. It uses Dynamic Programming.
3. Each item can be selected only once.
4. Items cannot be divided.
5. It uses a 2D DP table.
6. `max()` selects the better option.
7. Time complexity = O(n × W).
8. Space complexity = O(n × W).

Test your understanding

DAA Viva Practice

0/4 answered

1. Which technique is used in 0/1 Knapsack?

○ Greedy
○ Dynamic Programming
○ Binary Search
○ BFS

Correct answer: Dynamic Programming

Dynamic Programming stores results of smaller subproblems.

2. Why is it called 0/1 Knapsack?

○ Two bags are used
○ Items have two choices: select or reject
○ Capacity is always 1
○ Only two items are allowed

Correct answer: Items have two choices: select or reject

Each item is either selected (1) or not selected (0).

3. What is the time complexity of this implementation?

○ O(n)
○ O(n log n)
○ O(n × W)
○ $O(2^n)$

Correct answer: O(n × W)

Two nested loops run n and W times.

4. What does dp[n][W] represent?

○ Minimum weight
○ Number of items
○ Maximum value using all items within capacity W
○ Total weight of all items

Correct answer: Maximum value using all items within capacity W

The bottom-right cell stores the final maximum achievable value.

```
// Code-A5 (N-Queen)
#include
#include
using namespace std;
// Function to print the board
void printBoard(vector<vector>& board, int n) {
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++)
cout << board[i][j] << " ";
cout << endl;
}
cout << endl;
}
// Function to check if placing a queen is safe
bool isSafe(vector<vector>& board, int row, int col, int n) {
// Check column
for (int i = 0; i < row; i++)
if (board[i][col]) return false;
// Check upper left diagonal
for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i–, j–)
if (board[i][j]) return false;
// Check upper right diagonal
for (int i = row - 1, j = col + 1; i >= 0 && j < n; i–, j++)
if (board[i][j]) return false;
return true;
}
// Backtracking function
bool solveNQueens(vector<vector>& board, int row, int n) {
if (row == n) {
printBoard(board, n); // Print one valid arrangement
return true;
}
for (int col = 0; col < n; col++) {
if (isSafe(board, row, col, n)) {
board[row][col] = 1; // Place queen
solveNQueens(board, row + 1, n);
board[row][col] = 0; // Backtrack
}
}
return false;
}
int main() {
int n, firstRow, firstCol;
cout <<"Enter size of board (N): ";
cin >> n;
```

```
vector<vector> board(n, vector(n, 0));
cout <<"Enter position of first queen (row and column index starting from 0): ";
cin >> firstRow >> firstCol;
// Place first queen
board[firstRow][firstCol] = 1;
cout <<"\nAll possible solutions:\n";
solveNQueens(board, 0, n); // Start solving from row 0
return 0;
}
// SAMPLE OUTPUT
/
$ ./a.out
* Enter size of board (N): 4
* Enter position of first queen (row and column index starting from 0): 0 0

All possible solutions:
* 0 1 0 0
* 0 0 0 1
* 1 0 0 0
* 0 0 1 0

 0 0 1 0
* 1 0 0 0
* 0 0 0 1
* 0 1 0 0
*/
```

This C++ program implements the **N-Queens Problem using the Backtracking Algorithm.**

It places N queens on an N × N chessboard so that no two queens attack each other.

Let's understand it line by line in simple language for your DAA practical and viva.

1. What is the N-Queens Problem?

The N-Queens Problem is a famous problem in computer science where we have to place N queens on an N × N chessboard.

Rules:

- No two queens should be in the same row.
- No two queens should be in the same column.
- No two queens should be on the same diagonal.

For example, if N = 4, we have to place 4 queens on a 4 × 4 chessboard.

Which algorithm is used?

Backtracking Algorithm.

2. Explanation of the code

A. Header files

```
#include <iostream>
#include <vector>
using namespace std;
```

- `iostream`: Used for input and output (`cin`, `cout`).
- `vector`: Used to create and store the chessboard.
- `using namespace std`: Avoids writing `std::` repeatedly.

B. Print board function

```
void printBoard(vector<vector<int>>& board, int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            cout << board[i][j] << " ";

        cout << endl;
    }
    cout << endl;
}
```

This function prints the chessboard.

Explanation:

- `board`: Represents the chessboard.
- `n`: Represents the size of the board.
- Outer `for` loop: Goes through each row.
- Inner `for` loop: Goes through each column.
- `cout << board[i][j]`: Prints the value of each cell.

Meaning of 0 and 1:

Value	Meaning
0	Empty position
1	Queen is placed

C. isSafe() function

```
bool isSafe(vector<vector<int>>& board, int row, int col, int n)
```

This function checks whether placing a queen at a particular position is safe.

It returns:

- `true`: Position is safe.
- `false`: Position is not safe.

There are three important checks.

1. Check the column

```
for (int i = 0; i < row; i++)
    if (board[i][col])
        return false;
```

Checks whether another queen is already present in the same column above the current row.

If a queen is found, it returns false.

2. Check upper-left diagonal

```
for (int i = row - 1, j = col - 1;
     i >= 0 && j >= 0;
     i--, j--)
    if (board[i][j])
        return false;
```

Checks the upper-left diagonal for another queen.

Both row and column decrease by one in every iteration.

3. Check upper-right diagonal

```
for (int i = row - 1, j = col + 1;
     i >= 0 && j < n;
     i--, j++)
    if (board[i][j])
        return false;
```

Checks the upper-right diagonal for another queen.

- Row decreases.
- Column increases.

Finally:

```
return true;
```

If no queen is found in these three directions, the position is safe.

Important: We only check above the current row because this algorithm places queens row by row, starting from the top.

D. solveNQueens() function

```
bool solveNQueens(vector<vector<int>>& board, int row, int n)
```

This is the main backtracking function.

It tries to place queens in different columns of each row.

Step 1: Base condition

```
if (row == n) {
    printBoard(board, n);
    return true;
}
```

If `row == n`, it means all rows have been processed and N queens have been placed.

So:

- Print the valid chessboard.
- Return true.

Step 2: Try every column

```
for (int col = 0; col < n; col++) {
```

Checks every column in the current row.

Step 3: Check whether the position is safe

```
if (isSafe(board, row, col, n)) {
```

Calls the `isSafe()` function.

If the position is safe, we can place a queen there.

Step 4: Place the queen

```
board[row][col] = 1;
```

Places a queen at the selected position.

Step 5: Recursive call

```
solveNQueens(board, row + 1, n);
```

Moves to the next row and tries to place another queen.

Step 6: Backtracking

```
board[row][col] = 0;
```

Removes the queen from the current position.

Why?

Because the program needs to try other possible positions in the same row.

This process of placing a queen, exploring the next row and removing the queen when returning is called **Backtracking**.

E. Main function

```
int main() {
    int n, firstRow, firstCol;
```

Declares:

- `n`: Size of the chessboard.
- `firstRow`: Row position of the first queen.
- `firstCol`: Column position of the first queen.

Taking board size:

```
cout << "Enter size of board (N): ";
cin >> n;
```

Accepts the size of the chessboard.

Creating the chessboard:

```
vector<vector<int>> board(n, vector<int>(n, 0));
```

Creates an N × N chessboard and initializes every cell with zero.

Taking first queen position:

```
cin >> firstRow >> firstCol;
```

Accepts the row and column of the first queen.

Placing the first queen:

```
board[firstRow][firstCol] = 1;
```

Places the first queen at the position entered by the user.

Calling the backtracking function:

```
solveNQueens(board, 0, n);
```

Starts the solving process from row 0.

3. Dry run of the program

Input:

```
Enter size of board (N): 4

Enter position of first queen:
0 0
```

The first queen is fixed at row 0, column 0.

The program tries different positions for the remaining queens.

Solution 1

```
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

Solution 2

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

Note: There is an important mistake in the given code and sample output.

Your code places the first queen at `(0,0)`, but then calls:

```
solveNQueens(board, 0, n);
```

The function starts again from row 0 and does not check whether that row already contains a queen. Therefore, it can overwrite or ignore the initially fixed queen.

To correctly solve the problem with the first queen fixed, change:

```
solveNQueens(board, 0, n);
```

to:

```
solveNQueens(board, 1, n);
```

This starts placing the remaining queens from row 1.

Also, the `solveNQueens()` function prints every solution but does not use the returned `true` value to stop searching. This is intentional because the program wants to print all solutions.

4. Time and space complexity

Complexity	Value
Time complexity	O(N!) approximately
Space complexity	$O(N^2)$ for the board

Why?

- The algorithm tries different column positions for each row.
- Backtracking eliminates unsafe positions.
- The chessboard requires N × N memory.

The commonly quoted worst-case search complexity is O(N!), ignoring the additional safety-check cost.

5. Important viva questions and answers

Q1. What is the N-Queens Problem?

Ans: It is a problem of placing N queens on an N × N chessboard so that no two queens attack each other.

Q2. Which algorithm is used?

Ans: Backtracking Algorithm.

Q3. What is Backtracking?

Ans: Backtracking is an algorithmic technique that tries a possible solution and goes back if that solution cannot lead to a valid answer.

Q4. How does a queen move in chess?

Ans: A queen can move horizontally, vertically and diagonally.

Q5. What does 0 represent in the board?

Ans: An empty position.

Q6. What does 1 represent?

Ans: A position occupied by a queen.

Q7. What is the purpose of isSafe()?

Ans: It checks whether a queen can be placed safely without attacking another queen.

Q8. Why do we check only the upper rows?

Ans: Because queens are placed row by row from top to bottom. No queens have been placed in the lower rows yet.

Q9. What is the purpose of `board[row][col] = 0`?

Ans: It removes the queen so that the algorithm can try another possible position. This is called backtracking.

Q10. What is the base condition?

Ans: `if (row == n)`, which means all N queens have been successfully placed.

Q11. What happens when N = 4?

Ans: There are two valid arrangements for the standard 4-Queens problem.

Q12. What is the time complexity?

Ans: Approximately O(N!) for the backtracking search.

Q13. Can two queens be in the same row?

Ans: No.

Q14. Why do we use recursion?

Ans: To move from one row to the next and explore possible queen positions.

---

6. Important points to remember for practical viva

1. N-Queens is a constraint satisfaction problem.
2. It uses Backtracking.
3. One queen is placed in each row.
4. No two queens can share a row, column or diagonal.
5. `isSafe()` checks whether a position is valid.
6. `board[row][col] = 1` places a queen.
7. `board[row][col] = 0` removes a queen.
8. Time complexity is approximately O(N!).
9. Space complexity for the board is $O(N^2)$.

Test your understanding

DAA Viva Practice

0/4 answered

1. Which algorithm is used in N-Queens?

○ Greedy
○ Dynamic Programming
○ Backtracking
○ Binary Search

Correct answer: Backtracking

Backtracking explores possible queen positions and reverses unsafe choices.

2. What does 1 represent in the chessboard?

○ Empty cell
○ Queen is placed
○ Invalid position
○ Row number

Correct answer: Queen is placed

The program uses 1 for a queen and 0 for an empty cell.

3. What does isSafe() check?

○ Board size
○ Queen’s value
○ Whether a position is safe
○ Number of solutions

Correct answer: Whether a position is safe

It checks the column and both upper diagonals.

4. What is the purpose of board[row][col] = 0?

○ Place a queen
○ Stop the program
○ Backtrack and remove the queen
○ Print the board

Correct answer: Backtrack and remove the queen

It removes the previously placed queen so another position can be tested.